

Signed p -adic Residual Encodings of Finite-Domain All-Different Systems

with a Sudoku Case Study

Greg Baker

School of Computing, Australian National University

Canberra, Australia

greg.baker@anu.edu.au

ORCID: 0000-0002-6910-3011

Abstract

We give a mechanical translation from finite-domain all-different graphs and CNF formulae to signed, weighted p -adic regression data. Positive observations with sufficiently large weights restrict the model coefficients to allowed finite values; negative observations with smaller weights reward unequal endpoints or satisfied clauses. For a suitable prime, each residual norm on the intended finite domain is either zero or one, so minimising the regression objective is exactly minimising graph conflicts or maximising satisfied clauses. A three-variable worked example displays the complete dataframe and recovers its list-colourings by enumerating the finitely many zero-intercept hyperplanes determined by domain-pinning rows. A two-variable CNF example gives the same calculation for clauses. Standard Sudoku then supplies an 81-variable case study. We prove a general compiler theorem and provide a client-side implementation that exposes the generated dataframe and loss arithmetic. Different false clauses can also be assigned different polynomial labels, but this retains information already available from the clause list and gives no evident optimisation shortcut; we leave stronger uses of that encoding to future work.

Keywords: p -adic regression; signed residual objectives; all-different constraints; list colouring; Sudoku

1 p -adic linear regression

Fix a prime p . For a nonzero integer n , let $v_p(n)$ be the exponent of p in its factorisation and define

$$|n|_p = p^{-v_p(n)}, \quad |0|_p = 0.$$

This norm makes numbers close when their difference is divisible by a large power of p . Linear regression using sums of squared residuals needs little introduction: the familiar loss derives from the Euclidean distance formula, and weighted variants, including negatively weighted observations, are well known. Euclidean distance is not the only distance that can be used. In affine p -adic linear regression, an observation with feature vector u and target b has residual $u^\top x - b$, and a weighted objective sums terms of the form

$$\gamma |u^\top x - b|_p, \quad \gamma > 0.$$

This is a weighted p -adic linear-regression objective. Earlier results establish geometric and complexity properties of this positive-weight problem [1, 2]. We find no evidence that p -adic linear regression with negative weights has previously been studied. We demonstrate that it gives a useful representation of constraint-satisfaction problems such as Sudoku: source forms can be transformed mechanically into datasets for p -adic linear-regression training, and a resulting trained model can be transformed mechanically into a solution of the CSP.

1.1 An example

We begin with an example rather than the general theorem. Figure 1 has three variables. Each variable must take a value from its displayed domain, and adjacent variables must be different. The assignment $x = (1, 2, 3)$ is one solution.

For this example take $p = 5$. The only fact about the 5-adic norm needed at first is that $|0|_5 = 0$, while every nonzero integer residual that occurs below has norm 1. A dataframe row consists of a feature vector u , target b , and signed weight w ; at coefficient vector x it contributes

$$w |u^\top x - b|_5$$

to the objective. Positive rows of weight 3 pin each coefficient to its allowed values. Negative rows of weight -1 reward unequal endpoints.

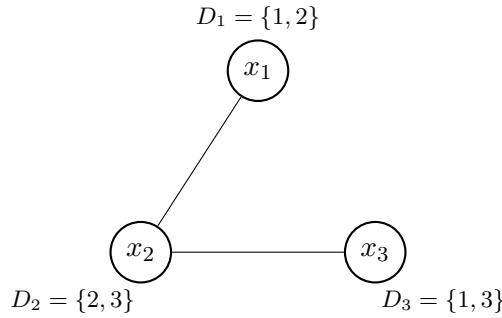


Figure 1: A tiny list-colouring instance. The assignment $x_1 = 1$, $x_2 = 2$, $x_3 = 3$ respects the domains and makes every edge unequal.

Row	x_1	x_2	x_3	Target	Signed weight	Role
P_1	1	0	0	1	+3	domain well
P_2	1	0	0	2	+3	domain well
P_3	0	1	0	2	+3	domain well
P_4	0	1	0	3	+3	domain well
P_5	0	0	1	1	+3	domain well
P_6	0	0	1	3	+3	domain well
E_1	1	-1	0	0	-1	reward $x_1 \neq x_2$
E_2	0	1	-1	0	-1	reward $x_2 \neq x_3$

Table 1: Dataframe view of Figure 2. The three coefficient columns form the feature vector, and each row contributes its signed weight times the 5-adic norm of feature dot coefficient minus target.

Table 1 gives the eight observations in dataframe form, with one column for each model coefficient. Figure 2 then shows the same observations in compact tuple notation. Each tuple (σ, u, b, γ) has signed weight $w = \sigma\gamma$.

Row	Type	Synthetic observation	Role
P_1	positive	$(+1, e_1, 1, 3)$	pin x_1 to D_1
P_2	positive	$(+1, e_1, 2, 3)$	pin x_1 to D_1
P_3	positive	$(+1, e_2, 2, 3)$	pin x_2 to D_2
P_4	positive	$(+1, e_2, 3, 3)$	pin x_2 to D_2
P_5	positive	$(+1, e_3, 1, 3)$	pin x_3 to D_3
P_6	positive	$(+1, e_3, 3, 3)$	pin x_3 to D_3
E_1	negative	$(-1, e_1 - e_2, 0, 1)$	reward $x_1 \neq x_2$
E_2	negative	$(-1, e_2 - e_3, 0, 1)$	reward $x_2 \neq x_3$

Figure 2: Synthetic dataset for the toy list-colouring instance in Figure 1. Here e_1, e_2, e_3 denote the standard basis vectors of $\mathbb{Z}^3$.

The zero-intercept model is the hyperplane

$$b = u^\top x = u_1 x_1 + u_2 x_2 + u_3 x_3.$$

The general theorem below guarantees that at least one global minimum occurs among the finitely many domain-respecting candidates.

On a domain-respecting vector the six positive rows contribute 9, while each unequal edge subtracts one. Thus

$$L_{\text{toy}}(x) = 9 - \mathbf{1}_{x_1 \neq x_2} - \mathbf{1}_{x_2 \neq x_3}.$$

Brute-force evaluation gives Table 2.

Coefficient vector x	$L_{\text{toy}}(x)$	Proper list-colouring?
$(1, 2, 1)$	7	yes
$(1, 2, 3)$	7	yes
$(1, 3, 1)$	7	yes
$(1, 3, 3)$	8	no
$(2, 2, 1)$	8	no
$(2, 2, 3)$	8	no
$(2, 3, 1)$	7	yes
$(2, 3, 3)$	8	no

Table 2: All eight hyperplanes determined by the domain-pinning rows.

The minimum-loss coefficient vectors are exactly the valid assignments of Figure 1; in particular, $x = (1, 2, 3)$ is recovered as a regression optimum.

1.2 p -adic linear regression can express CNF statements as well

Consider the CNF expression in the two Boolean variables z_1 and z_2 :

$$\Phi = (z_1) \wedge (\neg z_1 \vee z_2),$$

What values of z_1 and z_2 make Φ true? Encode $x_i = 0$ as $z_i = \text{true}$ and $x_i = 1$ as $z_i = \text{false}$. A clause is false at one affine equality, so a negative row rewards every coefficient vector away from that equality. The first clause is false when $x_1 = 1$, giving the residual $x_1 - 1$. The second is false when $(x_1, x_2) = (0, 1)$, giving $-x_1 + x_2 - 1$. With $p = 5$ and Boolean-pinning weight 3, the complete dataframe is Table 3.

Row	x_1	x_2	Target	Signed weight	Role
C_1	1	0	1	-1	reward (z_1)
C_2	-1	1	1	-1	reward ($\neg z_1 \vee z_2$)
$U_{1,0}$	1	0	0	+3	Boolean well
$U_{1,1}$	1	0	1	+3	Boolean well
$U_{2,0}$	0	1	0	+3	Boolean well
$U_{2,1}$	0	1	1	+3	Boolean well

Table 3: Signed regression dataframe compiled from the two-clause formula Φ . Constraint rows reward nonzero residuals; well rows force Boolean coefficients.

Choosing one well for x_1 and one for x_2 enumerates the four zero-intercept hyperplanes through the corresponding two basis rows. On Boolean coefficients the wells contribute 6, so the loss is $6 - \text{sat}_\Phi(x)$:

x	(z_1, z_2)	Clauses satisfied	Loss
(0, 0)	(true, true)	2	4
(0, 1)	(true, false)	1	5
(1, 0)	(false, true)	1	5
(1, 1)	(false, false)	1	5

Table 4: Brute-force regression for Φ .

The unique minimum has coefficients $x = (0, 0)$, which decode to the satisfying assignment $z_1 = z_2 = \text{true}$.

1.3 Polynomial-valued false labels

The preceding CNF gives a concrete polynomial-valued encoding. Let $V_{C_i}(x)$ be 1 when clause C_i is false and 0 when it is true, and take

$$P_{C_1}(T) = T, \quad P_{C_2}(T) = T + 1.$$

Form

$$R_x(T) = \prod_{i=1}^2 (1 - V_{C_i}(x) + V_{C_i}(x)P_{C_i}(T)) = \prod_{C_i \text{ false under } x} P_{C_i}(T).$$

Factoring R_x recovers the false-clause labels. Degree also defines a non-Archimedean absolute value on the rational-function field: for any fixed $c > 1$,

$$\left| \frac{f}{g} \right|_\infty = c^{\deg f - \deg g}, \quad |0|_\infty = 0.$$

Thus minimising $\deg R_x$ is equivalent to minimising this degree-based ultrametric loss. For the four assignments,

x	$R_x(T)$	$\deg R_x$
(0, 0)	1	0
(0, 1)	$T + 1$	1
(1, 0)	T	1
(1, 1)	T	1

so minimising $\deg R_x$ gives the same unique satisfying assignment as Table 4. The degree is exactly the number of false clauses, while factorisation retains their identities. Equivalently, each Boolean failure indicator can be written as the multilinear product of the literal-failure indicators, giving one positive nonlinear residual per clause [3].

We do not use this representation in the algorithm or the browser. Constructing it still requires evaluating the clauses, its factors reproduce the violation list already available from that evaluation, and its degree discards the extra labels again. It may be useful in a setting where polynomial-valued certificates are transmitted or manipulated independently, but no such use is established here. We therefore record it only as a possible direction for future research.

2 A general all-different residual-objective template

The worked examples can now be put on a general footing. We work in the p -adic integers $\mathbb{Z}_p$, where every norm is at most 1. If $p > q$, the difference between any two distinct values in $\{1, \dots, q\}$ has norm 1. For comparison, the discrete metric $d_{\text{disc}}(s, t) := \mathbf{1}_{s \neq t}$ is also an ultrametric; a discrete version of the construction is stated after the main theorem.

Many finite-domain all-different systems can be viewed as list-colouring problems. Each variable x_i has a finite allowed set D_i , and an edge $\{i, j\}$ requires $x_i \neq x_j$. An all-different constraint of arity r contributes the $\binom{r}{2}$ pairwise edges of its primal-graph clique.

Definition 1 (Signed synthetic p -adic residual dataset). *For parameters $x \in \mathbb{Z}_p^n$, a signed weighted affine observation is a quadruple (σ, u, b, γ) , where $\sigma \in \{+1, -1\}$ is the sign, $u \in \mathbb{Z}^n$ is the feature vector, $b \in \mathbb{Z}$ is the target, and $\gamma > 0$ is the weight. It contributes the term*

$$\sigma\gamma |u^\top x - b|_p$$

to the loss; formally, this is the p -adic analogue of a weighted Euclidean residual term.

Since $u \in \mathbb{Z}^n$, $b \in \mathbb{Z}$, and $x \in \mathbb{Z}_p^n$, each residual lies in $\mathbb{Z}_p$ and has norm at most 1. Every finite signed objective considered on this domain is therefore bounded. An observation is *positive* when $\sigma = +1$ and *negative* when $\sigma = -1$; positive observations will pin coefficients and negative observations will supply bounded rewards.

Theorem 1 (Finite-domain signed affine compiler template). *Fix a prime p . Let $D_i \subseteq \mathbb{Z}$ be finite nonempty sets whose elements are pairwise incongruent modulo p , and put*

$$D := \prod_{i=1}^n D_i \subseteq \mathbb{Z}_p^n.$$

For each coordinate i , choose a pinning weight $\lambda_i > 0$. Let $\mathcal{T}$ be a finite family of signed affine observations

$$(\sigma_\ell, u_\ell, b_\ell, \gamma_\ell), \quad \sigma_\ell \in \{\pm 1\}, \quad u_\ell \in \mathbb{Z}^n, \quad b_\ell \in \mathbb{Z}, \quad \gamma_\ell > 0,$$

and define

$$L(x) := \sum_{i=1}^n \lambda_i \sum_{a \in D_i} |x_i - a|_p + \sum_{\ell \in \mathcal{T}} \sigma_\ell \gamma_\ell |u_\ell^\top x - b_\ell|_p.$$

Assume that, for every coordinate i ,

$$\lambda_i > \sum_{\ell: (u_\ell)_i \neq 0} \gamma_\ell,$$

and that, for every $x \in D$ and every $\ell \in \mathcal{T}$, the residual $u_\ell^\top x - b_\ell$ is either 0 or a p -adic unit. Then L attains its global minimum on D , and every global minimiser lies in D . On D , L is a constant plus a finite signed objective whose interaction terms have values 0 and $\sigma_\ell \gamma_\ell$.

Proof. Suppose $x_i \notin D_i$. If x_i is congruent modulo p to an element of D_i , let b be that element; otherwise choose any $b \in D_i$. Let x' be obtained from x by replacing x_i with b .

The i -th unary sum decreases by at least $\lambda_i |x_i - b|_p$. Indeed, if $x_i \equiv b \pmod{p}$, then the only changed unary contribution below 1 is $|x_i - b|_p$. If x_i is congruent to no element of D_i , then $|x_i - a|_p = 1$ for all $a \in D_i$, while the unary sum at b has one zero term and all other terms equal to 1; in this case $|x_i - b|_p = 1$.

For an interaction term with $(u_\ell)_i \neq 0$, write $r'_\ell = u_\ell^\top x' - b_\ell$. Then

$$u_\ell^\top x - b_\ell = r'_\ell + (u_\ell)_i (x_i - b).$$

Since $(u_\ell)_i \in \mathbb{Z}$, $|(u_\ell)_i|_p \leq 1$, and the ultrametric inequality gives

$$||u_\ell^\top x - b_\ell|_p - |r'_\ell|_p| \leq |x_i - b|_p.$$

Thus the total interaction increase caused by changing coordinate i is at most

$$\sum_{\ell: (u_\ell)_i \neq 0} \gamma_\ell |x_i - b|_p.$$

By the assumed domination inequality, $L(x') < L(x)$. Repeating this coordinate-snapping step gives a point of D with smaller loss than any starting point outside D . Since D is finite, the global minimum is attained on D , and no global minimiser lies outside D .

On D , every unary sum is constant: exactly one term is zero and the rest are units. The second assumption says each interaction residual is either zero or a unit, so each signed interaction term is either 0 or $\sigma_\ell \gamma_\ell$. This proves the stated finite-domain form of $L|_D$. $\square$

Corollary 2 (Extension to $\mathbb{Q}_p^n$). *Under the assumptions of Theorem 1, view the same formula for L as an objective on $\mathbb{Q}_p^n$. It remains bounded below, attains its global minimum on D , and has no global minimiser outside D .*

Proof. Theorem 1 handles every coordinate in $\mathbb{Z}_p \setminus D_i$. If instead $|x_i|_p > 1$, then $|x_i - a|_p = |x_i|_p$ for every integer $a \in D_i$. Replacing x_i by any $b \in D_i$ decreases its unary sum by

$$\lambda_i (|D_i| |x_i|_p - (|D_i| - 1)) \geq \lambda_i |x_i|_p = \lambda_i |x_i - b|_p.$$

The interaction increase is bounded by $\sum_{\ell: (u_\ell)_i \neq 0} \gamma_\ell |x_i - b|_p$, exactly as in the theorem. The domination inequality therefore makes the replacement strictly improving. Snapping all coordinates first into $\mathbb{Z}_p$ and then into D proves the claim. In particular, every value of L is bounded below by the finite minimum attained on D . $\square$

Definition 2 (Synthetic dataset for list-colouring). *Let $V = \{1, \dots, n\}$, let $G = (V, E)$ be a finite graph, let $D_i \subseteq \{1, \dots, q\}$ be nonempty allowed sets, and fix a prime $p > q$. Define*

$$w_{ii} := 0, \quad w_{ij} := \begin{cases} 1, & \{i, j\} \in E, \\ 0, & \{i, j\} \notin E, \end{cases} \quad (i \neq j),$$

let $\Delta := \max_{i \in V} \sum_{j=1}^n w_{ij}$, and set $\lambda := 1 + \Delta$. Writing $e_1, \dots, e_n$ for the standard basis vectors of $\mathbb{Z}^n$, define

$$\mathcal{S}^+ := \{(+1, e_i, a, \lambda) : i \in V, a \in D_i\},$$

$$\mathcal{S}^- := \{(-1, e_i - e_j, 0, 1) : 1 \leq i < j \leq n, w_{ij} = 1\}.$$

Set $\mathcal{S} := \mathcal{S}^+ \sqcup \mathcal{S}^-$.

The theorem is stated for a simple graph, so $w_{ij} \in \{0, 1\}$. An edge-weighted version is obtained by replacing the unit negative observation on edge $\{i, j\}$ with weight $c_{ij} > 0$ and choosing the pinning weight larger than $\max_i \sum_j c_{ij}$, but the binary case is the one needed here.

Definition 3 (Loss associated with the list-colouring dataset). *For $\mathcal{S}$ as in Definition 2, define*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^n \sum_{a \in D_i} |x_i - a|_p - \sum_{1 \leq i < j \leq n} w_{ij} |x_i - x_j|_p.$$

This dataset has exactly $\sum_{i=1}^n |D_i|$ positive observations and $|E|$ negative observations, so it is computable in time $O(\sum_i |D_i| + |E|)$. The positive observations pin coordinates to allowed values, and the negative observations reward inequality across edges.

Each positive observation $(+1, e_i, a, \lambda)$ contributes $\lambda |x_i - a|_p$ to the loss, and each negative observation $(-1, e_i - e_j, 0, 1)$ contributes $-|x_i - x_j|_p$ to the loss. Summing them yields the displayed loss. The size bound and runtime are immediate from the explicit definitions.

The compiler template already implies that minimisers snap to the finite alphabet, because the pinning weight is larger than the degree of every vertex. The next proof records the stronger fact needed for all-different systems: after snapping, the objective is exactly a constant plus the edge-conflict count.

2.1 Correctness

Since $p > q$, the residues of $1, \dots, q$ are distinct modulo p . In particular, for any fixed $i \in V$ and any $t \in \mathbb{Z}_p$, there is at most one $a \in D_i$ with $t \equiv a \pmod{p}$.

First we record the simple observation that, on $\{1, \dots, q\}$, the p -adic norm detects inequality exactly.

Lemma 1 (Edge terms become inequality indicators on the domain). *If $x_i, x_j \in \{1, \dots, q\}$ and $p > q$, then*

$$|x_i - x_j|_p = \begin{cases} 0, & x_i = x_j, \\ 1, & x_i \neq x_j. \end{cases}$$

Proof. If $x_i = x_j$ then the difference is zero. If $x_i \neq x_j$, then $1 \leq |x_i - x_j| \leq q - 1 < p$, so $p \nmid (x_i - x_j)$ and therefore $|x_i - x_j|_p = 1$. $\square$

We next prove a digit-snapping lemma: if x minimises $L_{\mathcal{S}}$, then each coordinate x_i lies in D_i . The explicit unary-well formula makes that argument transparent.

Lemma 2 (Unary wells). *For $i \in V$, define*

$$U_i(t) := \sum_{a \in D_i} |t - a|_p \quad (t \in \mathbb{Z}_p).$$

Then $U_i(t) \geq |D_i| - 1$, with equality if and only if $t \in D_i$.

Proof. If $t \in D_i$, then one summand in $U_i(t)$ is 0, and the remaining $|D_i| - 1$ summands are 1, so $U_i(t) = |D_i| - 1$.

If $t \notin D_i$ but $t \equiv b \pmod{p}$ for some $b \in D_i$, then $0 < |t - b|_p < 1$, while for every $a \in D_i \setminus \{b\}$ we have $t - a \equiv b - a \not\equiv 0 \pmod{p}$, so $|t - a|_p = 1$. Hence

$$U_i(t) = (|D_i| - 1) + |t - b|_p > |D_i| - 1.$$

If t is congruent modulo p to no element of D_i , then $|t - a|_p = 1$ for all $a \in D_i$, so $U_i(t) = |D_i| > |D_i| - 1$. $\square$

Lemma 3 (Digit-snapping lemma). *Let $x \in \mathbb{Z}_p^n$. Suppose there exists $i \in V$ with $x_i \notin D_i$. Choose $b \in D_i$ as follows: if x_i is congruent modulo p to an element of D_i , take that unique element; otherwise choose any $b \in D_i$. Let x' be obtained from x by replacing x_i with b . Then*

$$L_{\mathcal{S}}(x') < L_{\mathcal{S}}(x).$$

Proof. The positive part of the loss changes by

$$\lambda \left(\sum_{a \in D_i} |b - a|_p - \sum_{a \in D_i} |x_i - a|_p \right).$$

There are two cases to consider.

If x_i is congruent modulo p to no element of D_i , then $|x_i - a|_p = 1$ for every $a \in D_i$, so $\sum_{a \in D_i} |x_i - a|_p = |D_i|$. On the other hand, one summand in $\sum_{a \in D_i} |b - a|_p$ is 0 and the remaining $|D_i| - 1$ summands are 1, so $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$. Hence the positive change is $-\lambda$. Since $|x_i - b|_p = 1$ in this case, this is at most $-\lambda|x_i - b|_p$.

If $x_i \equiv b \pmod{p}$ for some $b \in D_i$, then $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$, whereas

$$\sum_{a \in D_i} |x_i - a|_p = (|D_i| - 1) + |x_i - b|_p.$$

Hence the positive change is $-\lambda|x_i - b|_p$.

For each $j \in V$, the corresponding negative component changes by

$$w_{ij}(|x_i - x_j|_p - |b - x_j|_p).$$

By the ultrametric inequality,

$$|x_i - x_j|_p \leq \max(|x_i - b|_p, |b - x_j|_p),$$

so

$$|x_i - x_j|_p - |b - x_j|_p \leq |x_i - b|_p.$$

Summing over all $j \in V$,

$$L_{\mathcal{S}}(x') - L_{\mathcal{S}}(x) \leq \left(-\lambda + \sum_{j=1}^n w_{ij} \right) |x_i - b|_p \leq -|x_i - b|_p < 0.$$

□

On domain-respecting assignments, the loss is an additive constant plus the conflict count.

Theorem 3 (Correctness of the synthetic reduction). *For the dataset $\mathcal{S} = \mathcal{S}^+ \sqcup \mathcal{S}^-$ from Definition 2 and the loss from Definition 3, every global minimiser of $L_{\mathcal{S}}$ lies in $\prod_{i=1}^n D_i$. On $\prod_{i=1}^n D_i$,*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^n (|D_i| - 1) - |E| + \sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1}_{x_i = x_j}.$$

Consequently, the global minimisers are exactly the domain-respecting assignments that minimise the conflict sum $\sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1}_{x_i = x_j}$, or equivalently maximise the number of unequal edges. In particular, if the list-colouring instance $(G, (D_i)_{i=1}^n)$ is satisfiable, then this minimum conflict sum is 0, and the global minimisers are exactly its proper list-colourings.

Proof. By Lemma 3, any point with some coordinate outside its domain can be improved by snapping that coordinate into the domain. Therefore every global minimiser lies in $\prod_i D_i$. If $x \in \prod_i D_i$, then Lemma 2 gives $U_i(x_i) = |D_i| - 1$ for every i , and Lemma 1 gives

$$|x_i - x_j|_p = \mathbf{1}_{x_i \neq x_j} \quad (1 \leq i < j \leq n).$$

Since $\mathbf{1}_{x_i \neq x_j} = 1 - \mathbf{1}_{x_i = x_j}$ on every edge, substituting these identities into $L_{\mathcal{S}}$ yields the displayed formula. A global minimiser exists: $L_{\mathcal{S}}$ takes only finitely many values on the finite set $\prod_i D_i$,

and by the previous paragraph no point outside $\prod_i D_i$ can be a global minimiser, so the global minimum is attained on $\prod_i D_i$. There the displayed formula is an additive constant plus the conflict sum, so a domain-respecting assignment is a global minimiser if and only if it attains the least possible conflict sum; this is the stated “exactly” characterisation in both directions. In particular, if $(G, (D_i)_{i=1}^n)$ is satisfiable the least conflict sum is 0, and the global minimisers are exactly its proper list-colourings. $\square$

Corollary 4. *Any finite-domain all-different constraint system can be encoded, in polynomial time, as a signed weighted synthetic p -adic residual objective whose global minimisers are exactly the domain-respecting assignments with minimum conflict count. If the constraint system is satisfiable, these minimisers are exactly its satisfying assignments.*

Proof. Relabel the union of the explicitly listed domain values by $\{1, \dots, q\}$, preserving equality across domains. Here $q \leq \sum_i |D_i|$. Choose a prime $p > q$ (for $q > 1$, Bertrand’s postulate gives one below $2q$); because q is bounded by the explicit input length, scanning this interval with deterministic primality testing is polynomial in that input length. Form the primal graph: the vertices are the variables, and two variables are adjacent if they appear together in some all-different constraint. The satisfying assignments are exactly the proper list-colourings of this graph, with the variable domains as lists, and the minimum-conflict assignments are exactly the domain-respecting assignments minimising the corresponding deduplicated edge-conflict sum. Apply Definitions 2 and 3, and Theorem 3. $\square$

Discrete-metric variant. The same synthetic dataset also works with the discrete metric

$$d_{\text{disc}}(s, t) := \mathbf{1}_{s \neq t}.$$

If each residual $|u^\top x - b|_p$ is replaced by $d_{\text{disc}}(u^\top x, b)$, then the loss becomes

$$L_S^{\text{disc}}(x) = \lambda \sum_{i=1}^n \sum_{a \in D_i} \mathbf{1}_{x_i \neq a} - \sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1}_{x_i \neq x_j}.$$

The unary wells satisfy

$$U_i^{\text{disc}}(t) := \sum_{a \in D_i} \mathbf{1}_{t \neq a} = \begin{cases} |D_i| - 1, & t \in D_i, \\ |D_i|, & t \notin D_i. \end{cases}$$

Hence replacing any $x_i \notin D_i$ by a value $b \in D_i$ decreases the positive part by exactly λ , while the total increase in the negative part is at most $\sum_j w_{ij} \leq \Delta$. Since $\lambda = 1 + \Delta$, the same snapping argument goes through. Therefore the conclusions of Theorem 3 and Corollary 4 remain valid for the discrete metric as well. The p -adic case remains the main one because it realises the same indicator behaviour inside the affine p -adic regression framework used throughout the paper.

Boolean clause rewards. Let

$$\Phi = \bigwedge_{r=1}^m C_r$$

be a CNF formula on variables $z_1, \dots, z_n$, with every clause of width at most k , and encode truth values by

$$x_i = 0 \iff z_i \text{ is true}, \quad x_i = 1 \iff z_i \text{ is false.}$$

For a literal λ on variable z_i , define its failure indicator by

$$\delta(\lambda; x) = \begin{cases} x_i, & \lambda = z_i, \\ 1 - x_i, & \lambda = \neg z_i. \end{cases}$$

For a clause $C = (\lambda_1 \vee \dots \vee \lambda_s)$, where $s \leq k$,

$$C(x) \text{ is false} \iff \sum_{j=1}^s \delta(\lambda_j; x) = s.$$

After moving constants to the right-hand side, this means that there are $u_C \in \mathbb{Z}^n$ and $t_C \in \mathbb{Z}$ such that

$$C(x) \text{ is true} \iff u_C^\top x \neq t_C.$$

Here u_C records the ± 1 literal coefficients, and $t_C = s - \nu_C$, where ν_C is the number of negated literals in the clause. The translation is mechanical: a positive literal contributes $+x_i$, a negated literal contributes $-x_i$, and the forbidden right-hand side is $s - \nu_C$. For example,

$$(z_1 \vee z_2 \vee \neg z_3) \rightsquigarrow x_1 + x_2 - x_3 \neq 2.$$

If $p > k$ and $x \in \{0, 1\}^n$, then $u_C^\top x - t_C$ is 0 when C is false and belongs to $\{-1, \dots, -s\}$ when C is true, so

$$|u_C^\top x - t_C|_p = \mathbf{1}_{C(x) \text{ is true}}.$$

Thus a negative observation $(-1, u_C, t_C, 1)$ rewards satisfaction of the clause. If Δ is the maximum number of clauses containing any variable and $\alpha > \Delta$, then

$$L_\Phi(x) = \alpha \sum_{i=1}^n (|x_i|_p + |x_i - 1|_p) - \sum_{C \in \Phi} |u_C^\top x - t_C|_p$$

has the same snapping property as before: every global minimiser lies in $\{0, 1\}^n$, and on Boolean assignments

$$L_\Phi(x) = \alpha n - \text{sat}_\Phi(x).$$

So the global minimisers are exactly the assignments that maximise the number of satisfied clauses. Unlike the earlier ordinary summed 2-adic MAX-CUT reduction [2], this construction uses a signed objective; its 3-CNF specialisation assumes $p > 3$.

Corollary 5 (NP-hardness at a fixed prime). *Globally minimising signed affine p -adic residual objectives is NP-hard already for the fixed prime $p = 5$.*

Proof. Apply the construction above to a 3-CNF formula, with $p = 5$ and $\alpha > \Delta$. Its minimum equals $\alpha n - m$ if and only if all m clauses can be satisfied. A global optimiser would therefore decide 3-SAT, so the optimisation problem is NP-hard. $\square$

Corollary 6 (NP-hardness for the polynomial degree metric). *Given pairwise coprime linear false-clause labels, minimising the degree-based ultrametric loss $\deg R_x$ over Boolean assignments is NP-hard.*

Proof. Apply the construction of Section 1.3 to a 3-CNF formula, assigning each clause a distinct linear polynomial. Then $\deg R_x$ is exactly the number of false clauses. In particular, its minimum is 0 if and only if the formula is satisfiable, so a global optimiser would decide 3-SAT. $\square$

Signed versus positive complement encodings. The signed construction is compact rather than uniquely expressive. The same clause can be encoded positively through its failure sum

$$s_C(x) := \sum_{j=1}^s \delta(\lambda_j; x)$$

and the allowed set $S_C = \{0, \dots, s-1\}$. For $S \subseteq \{0, \dots, s\}$, set

$$W_S(t) = \sum_{a \in S} |t - a|_p.$$

When $p > k$ and $t \in \{0, \dots, s\}$, the positive term $W_{S_C}(t)$ equals $s-1$ on the satisfying values $0, \dots, s-1$ and s on the forbidden value s . It therefore encodes exactly the same clause penalty using s positive observations. By contrast, the signed encoding uses one negative observation for the single forbidden affine equality. Filling the missing positive well at s makes the positive sum constant, which illustrates the complement relationship. The contribution of the signed form is the direct one-row representation of a small forbidden set, together with the coordinatewise domination theorem that keeps its negative reward bounded and snaps variables to their domains.

3 Minimum conflicts and Potts energies

3.1 Unsatisfiable instances

The characterisation holds even when the instance has no proper list-colouring: there the global minimisers are the domain-respecting assignments of least conflict count. For example, take the triangle K_3 with all three lists equal to $\{1, 2\}$. No proper 2-colouring of K_3 exists, so every domain-respecting assignment leaves at least one edge monochromatic. Here $\Delta = 2$, so $\lambda = 3$, and by Theorem 3 the global minimisers are exactly the assignments with a single conflict, at loss $\lambda \sum_i (|D_i| - 1) - |E| + 1 = 3 \cdot 3 - 3 + 1 = 7$. This Max-CSP regime is distinct from the satisfiable case.

If a best-effort solution should treat some constraints as more important than others, assign them correspondingly higher or lower losses. Replacing the unit reward on an edge e by a weight $w_e > 0$, and raising each pinning weight λ_i above the total weight incident on coordinate i , gives the weighted minimum-conflict objective by the same domination argument.

3.2 Potts energies

On domain-respecting states the objective equals, up to the additive constant $\lambda \sum_i (|D_i| - 1) - |E|$, the number of monochromatic edges $\sum_{\{i,j\} \in E} \mathbf{1}_{x_i=x_j}$, which is the energy of an antiferromagnetic q -state Potts model on G with the lists D_i as local state constraints. The construction does not alter that energy landscape; its content is the representation of this Potts/Max-CSP energy as a signed affine p -adic residual objective on the native finite-domain variables.

4 Sudoku as a special case

Sudoku has standard exact-cover, constraint-programming, and SAT formulations [9, 10, 11], and puzzle variants have their own complexity theory [12]. The construction below is not meant to compete with those solvers. Sudoku is used because its variables, domains, and all-different constraints are small enough to write out explicitly. We then perform experiments with p -adic machine-learning training methods and demonstrate that these techniques generally succeed at solving the Sudoku-devil instances.

4.1 Sudoku

A (standard) Sudoku puzzle consists of a partially filled 9×9 grid. The task is to assign a digit in $\{1, \dots, 9\}$ to each empty cell such that:

1. each row contains all digits $1, \dots, 9$ exactly once;

2. each column contains all digits $1, \dots, 9$ exactly once; and
3. each 3×3 box contains all digits $1, \dots, 9$ exactly once.

Equivalently, in each unit (row, column, box) all entries are *pairwise distinct* and each entry lies in $\{1, \dots, 9\}$.

Standard Sudoku fits Corollary 4 directly. The vertices are the 81 cells, two vertices are adjacent when the corresponding cells share a row, column, or 3×3 box, unclued cells have domain $\{1, \dots, 9\}$, and clue cells have singleton domains. A proper list-colouring of this graph is exactly a valid Sudoku completion.

The Sudoku facts below are direct corollaries of the unary-well and edge-indicator lemmas. We state them before writing the concrete objectives.

4.2 Positive pinning: snapping coefficients to digits

Let $D = \{1, 2, \dots, 9\} \subset \mathbb{Z}_p$, and write the Sudoku variables as a vector

$$x \in \mathbb{Z}_p^{81},$$

with one coefficient per cell.

Definition 4 (Sudoku digit-snapping penalty). *For $p > 9$, define the digit-snapping penalty*

$$U_D(t) := \sum_{k=1}^9 |t - k|_p.$$

By Lemma 2 applied to the common domain D , one has $U_D(t) \geq 8$ for every $t \in \mathbb{Z}_p$, with equality if and only if $t \in D$.

The unary term is the nine-valued specialisation of the arbitrary-residue positive wells in Mihara’s general forcing theorem [15]: the observations $t = k$ for all $k \in D$ create a multi-well objective with minima exactly at D . The additional signed terms below reward pairwise inequality and produce the Sudoku/list-colouring objective.

4.3 Negative signed residual terms: rewarding inequality via p-adic norms

The remaining Sudoku constraints are that entries in each row/column/box are pairwise unequal. Pairwise inequality can be encoded using differences $x_i - x_j$.

Corollary 7 (Sudoku differences as inequality indicators). *Let $p > 9$ and $a, b \in D$. Then*

$$|a - b|_p = \begin{cases} 0 & a = b, \\ 1 & a \neq b. \end{cases}$$

Proof. This is Lemma 1 with $q = 9$. □

4.4 Minimal and expanded residual objectives

Let $\mathcal{P}$ be the set of cell-pairs (i, j) that share a unit (same row, same column, or same box). Let $\mathcal{C}$ be the set of given clues; for $i \in \mathcal{C}$ let $g_i \in D$ be the fixed digit.

Specialising Definitions 2 and 3, together with Theorem 3, to the Sudoku graph gives the minimal synthetic dataset

$$L_{\min}(x) = \alpha \sum_{i \notin \mathcal{C}} \sum_{k=1}^9 |x_i - k|_p + \alpha \sum_{i \in \mathcal{C}} |x_i - g_i|_p - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \quad (1)$$

A separate coefficient on the pairwise term is unnecessary here: any positive edge weight can be scaled out, so we normalise that coefficient to 1. Thus α itself plays the role of the general pinning weight λ , and any choice $\alpha > 20$ satisfies the domination condition from Theorem 3. The minimal dataset therefore contains $9(81 - |\mathcal{C}|) + |\mathcal{C}| = 729 - 8|\mathcal{C}|$ positive observations and 810 negative observations. This is the formulation used in the correctness statements below.

For exposition, we also use the redundant expanded objective

$$L(x) = \alpha \sum_{i=1}^{81} \sum_{k=1}^9 |x_i - k|_p + \alpha_{\text{clue}} \sum_{i \in \mathcal{C}} |x_i - g_i|_p - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \quad (2)$$

Equation (2) keeps the nine digit-well residuals visible even on clue cells and then adds the clue pins separately. It therefore contains $729 + |\mathcal{C}|$ positive observations and the same 810 negative observations. The first two sums are positive pinning terms, and the final sum is a negative pairwise reward term. Equivalently, counting peer pairs per unit gives $27 \binom{9}{2} = 972$ negative terms, but 162 of these pairs lie simultaneously in a row/column *and* a box, leaving 810 distinct pairs overall.

This expanded form needs an explicit clue-weight assumption. On a clue cell, the first sum in (2) has the same value 8α for every digit $x_i \in D$, so clue fidelity is enforced only by $\alpha_{\text{clue}} |x_i - g_i|_p$. The same domination argument applies to (2) when $\alpha_{\text{clue}} > 20$; the simplest choice is $\alpha_{\text{clue}} = \alpha = 21$. Under that extra condition, (2) is a redundant expanded form of (1).

Once x is digit-valued, the pairwise part is simply

$$- \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p = -|\mathcal{P}| + \sum_{(i,j) \in \mathcal{P}} \mathbf{1}_{x_i = x_j},$$

For the minimal objective on domain-respecting states, and for the expanded objective on digit-valued, clue-consistent states, the remaining terms are constant. In those regimes the loss differs from the *deduplicated* peer-conflict count only by an additive constant. The heuristics below do not optimise that deduplicated peer objective directly: on the row-permutation state space they use a duplicated unit-scope column/box surrogate that counts conflicts separately inside each column and each box. That surrogate is described in Section 6 and Appendix A.

Corollary 8 (Sudoku completion as a special case). *Let G_{Sud} be the graph on the 81 cells of a Sudoku puzzle, with an edge between two cells exactly when they share a row, column, or 3×3 box. For a puzzle with clue set $\mathcal{C}$ and clue digits g_i , define*

$$D_i := \begin{cases} \{g_i\}, & i \in \mathcal{C}, \\ \{1, \dots, 9\}, & i \notin \mathcal{C}. \end{cases}$$

Assume $\alpha > 20$. Then every global minimiser of the minimal objective (1) is a clue-respecting digit assignment that minimises the peer-conflict count. In particular, if the Sudoku instance is satisfiable, the global minimisers are exactly the valid completions of the puzzle.

Proof. Each row, column, and box of Sudoku forms a clique of size 9 in G_{Sud} . Thus a proper list-colouring assigns pairwise distinct digits in each unit. Since the available values are exactly $\{1, \dots, 9\}$, pairwise distinctness in a unit is equivalent to containing every digit exactly once. The singleton domains enforce the clues. Apply Theorem 3. $\square$

In the deduplicated peer graph, every cell has degree 20, so Theorem 3 requires $\alpha > 20$ for the minimal objective (1); $\alpha = 21$ is enough. For the expanded objective (2), the same claim requires $\alpha_{\text{clue}} > 20$, because the all-digit wells are constant on clue cells. If one instead counts row, column, and box overlaps separately, each cell has $8 + 8 + 8 = 24$ incident negative terms, and the safe unary bound becomes > 24 .

5 Representational parsimony

A standard SAT or exact-cover lift introduces Boolean variables

$$y_{r,c,d} \in \{0,1\},$$

with the intended meaning that cell (r, c) contains digit d . This produces $9 \times 9 \times 9 = 729$ indicators before any clues are applied. The p -adic construction instead uses an 81-cell digit space. This variable-level comparison is the main parsimony claim: one digit-valued parameter per semantic object rather than nine Boolean indicators per cell.

The minimal objective (1) is already a single scalar loss on that state space. The theorem-side expanded objective (2) contains 729 digit-snapping residuals, $|\mathcal{C}|$ clue residuals, and 810 pairwise residuals. A structured constraint-programming model with 81 cell variables and 27 *alldifferent* constraints is already semantically close to the puzzle, so the comparison here is not based on raw constraint count alone. The signed p -adic construction packages the all-different structure into one residual objective on the 81 cell variables.

Appendix C records an alternative encoding that maps digits to powers of two so that each row, column, or box contributes a single sum residual rather than pairwise terms. In local-search experiments on 50 benchmark puzzles, this encoding produced no successful solves: the search repeatedly stopped in local minima.

6 Computations using p -adic machine-learning techniques

We implemented two simple local-search procedures: a greedy row-swap search and a discrete Zubarev walk. Appendix A defines their loss, records the algorithms and a locality lemma for row swaps, and gives traces on a standard puzzle.

These are p -adic machine-learning techniques in this setting because they train the coefficient vector by local moves that reduce a loss derived from the signed p -adic residual objective. In particular, row-swap search is a discrete graph walk on the state graph of clue-consistent row permutations, in the same spirit as earlier graph-walk experiments.

On a small sample of 18 randomly carved puzzles, both heuristics solved every puzzle, giving 36 successful method-puzzle runs. The row-swap search used fewer steps on 14 of the 18 paired puzzles. Its median step counts were 537.5, 2784, and 7040 at 36, 30, and 26 clues, versus 4533.5, 8751.5, and 9911 for the Zubarev walk. Appendix B gives the experimental setup, full table, and a representative loss curve. These results show that general p -adic linear-regression heuristics can solve most of the sampled Sudoku instances in practice, but the sample is not a solver benchmark.

<https://padic-logic.symmachus.org>



Figure 3: The Sudoku route after a row-swap run reaches a valid completion. The native 81-cell grid, the 1,299-row signed objective summary, the theoretical-floor equality, and the zero-conflict search result remain visible together.

7 Related work and scope

7.1 Earlier regression and constraint models

From the constraint-programming side, the finite-domain objective is a valued-CSP or Max-CSP penalty function [4]. Replacing a global *alldifferent* constraint by a clique of pairwise inequalities preserves its solutions but not the stronger propagation of a matching-based global constraint [5]. On the finite domain, the list-colouring objective is the zero-temperature antiferromagnetic Potts energy with local list constraints [6]; related discrete problems are often mapped to Ising or QUBO objectives [7]. The local-search experiments are closest algorithmically to min-conflicts repair search [8]. Sudoku itself is routinely formulated using exact cover, SAT, or *alldifferent* constraints [9, 10, 11].

7.2 Why we could not use Mihara regression

Mihara’s forcing theorem [15] considers a product $C = \prod_i B_i$ of residue sets, a nonnegative base objective satisfying a concentration condition near C , and a positive unary forcing term whose weight M exceeds the base objective’s range on C . The forced objective has the same optima on C ; specialising the construction to affine residual data proves ordinary positive p -adic linear regression NP-hard for every fixed prime.

Mihara’s linear-regression algorithm is described in [14]. For his digitwise regression algorithm, the data are samples

$$(\vec{x}_i, y_i) \in \mathbb{Z}_p^D \times \mathbb{Z}_p$$

from a single hidden affine graph

$$y = \langle c, \vec{x} \rangle,$$

with digitwise noise. Writing

$$I_e = \{i : y_i - \langle c, \vec{x}_i \rangle \in p^e \mathbb{Z}_p\},$$

the method estimates $c \bmod p^E$ recursively: first solve a modulo- p linear-regression problem to estimate $c \bmod p$, subtract that digit, divide residuals by p , restrict to the samples in I_1 , and repeat. The probabilistic argument relies on a large, sufficiently random sample from the hidden affine graph, a small digitwise corruption rate such as

$$\frac{|I_e \setminus I_{e+1}|}{|I_e|} \leq r \ll \frac{1}{2},$$

	Mihara’s forcing theorem	Signed affine compiler here
Base terms	Nonnegative p -adic optimisation concentrated near a product of residue sets	Signed affine residuals that are zero or units on the target finite domain
Pinning condition	One forcing scale M above the base objective’s range on the finite set	Coordinatewise λ_i above the absolute incident interaction weight
Off-domain control	Concentration plus positive unary forcing	Ultrametric Lipschitz bound for positive and negative interaction rows
Finite-domain role	Preserve the base objective’s optima on C	Realise equality, inequality, and forbidden-hyperplane indicators exactly
Main consequence	Positive linear regression NP-hard for each fixed prime	Compact complement encoding and exact minimum-conflict characterisation

Table 5: Comparison with Mihara’s generalisation of Baker forcing [15]. The two results share finite-domain pinning but use different hypotheses to control the remaining objective.

and the condition that the noise-free samples have the right affine hull modulo p at each digit step.

Those assumptions do not match the Sudoku construction. In this paper the unknowns x_i are the cell values themselves, not coefficients of an observed response law. The residual rows are exact synthetic constraints chosen by the modeller: unary wells pin cells to allowed digits, while negative residuals $-|x_i - x_j|_p$ reward inequality across peer edges. After digit snapping, the problem is a finite-domain conflict-minimisation problem. There is no response vector Y , no hidden coefficient vector c to recover from random samples, and no large noise-free affine locus whose graph could be estimated digit by digit.

Mihara’s method also does not apply to the CNF construction. A clause becomes one *forbidden* affine equality

$$u_C^\top x = t_C,$$

and a satisfying assignment is rewarded because the residual is nonzero. Mihara’s modulo- p stage is designed to recover an affine equality satisfied by most noise-free samples. The Boolean CSP task asks for one point avoiding many clause-specific forbidden hyperplanes, or, in the Max-CSP case, avoiding as many as possible. Failed clauses are not random digitwise outliers from a single affine law; they are exact, structured constraints sharing variables.

This is not surprising: if Mihara’s method could quickly solve these instances, it would provide a polynomial-time solver for NP-hard problems.

8 Conclusion

Negative weights in Euclidean squared-loss regression are usually treated as anti-shrinkage terms in selected directions. Here signed residuals have a different role. Positive synthetic terms pin variables to a finite set; after that, negative p -adic terms are bounded rewards for inequality on domain-respecting states.

Sudoku is a finite-domain case study in which the variables, domains, and all-different constraints are explicit. All-different systems admit polynomial-time encodings whose global minimisers are exactly the domain-respecting minimum-conflict assignments, and in the satisfiable case exactly the satisfying assignments.

The same signed-residual construction is not limited to pairwise inequality. After Boolean pinning, every clause of width less than p becomes one forbidden affine equality, so a single signed term rewards its satisfaction. A positive-only affine encoding can represent the same

finite-domain penalty by listing all allowed residual values; the signed formulation’s advantage is the compact one-row representation of the forbidden complement within that affine class. Section 1.3 notes that different false clauses can be given different polynomial labels, but the resulting factorisation merely repackages the already-computed violation list. No optimisation or verification advantage is claimed for it.

The companion site in Figure 3 lets readers trace a Boolean CSP from clauses to forbidden affine equalities and a signed residual dataframe, or compile an editable Sudoku into its 1,299-row objective and watch search reach the theoretical floor.

A Heuristic details

A.1 Row-swap local search

To keep local updates cheap, the implementation does not minimise (2) directly. Instead, on the digit-valued, clue-consistent row-permutation state space it minimises the duplicated unit-scope column/box conflict count

$$H_{\text{cb}}(x) := \sum_{c=1}^9 \text{conf}(C_c; x) + \sum_{b=1}^9 \text{conf}(B_b; x),$$

where C_c and B_b are the cells in column c and box b , and

$$\text{conf}(U; x) := \sum_{d=1}^9 \binom{\#\{u \in U : x_u = d\}}{2}.$$

Because each row is kept as a permutation of $\{1, \dots, 9\}$, the corresponding row contribution is identically zero and is omitted. This duplicated unit-scope loss is close to, but not identical with, the deduplicated peer-conflict objective coming from $\mathcal{P}$: a conflicting pair that lies simultaneously in a column and a box is counted twice. For the seed-0 initialisation shown later in Figure 5, the deduplicated peer-conflict count is 47 whereas $H_{\text{cb}} = 56$, because nine conflicting pairs lie in both a column and a box.

The heuristics work directly on the digit-valued, clue-consistent row-permutation state space and minimise H_{cb} . Concretely, we initialise each row as a random permutation of $1, \dots, 9$ consistent with its clues, and then apply row-wise swaps that reduce H_{cb} . This is a block-local search on the integer state space.

Lemma 4 (Locality of row swaps). *Let x be a digit-valued, clue-consistent Sudoku state in which each row is a permutation of $1, \dots, 9$. Let ξ swap two non-clue entries in a single row r , at columns $c_1 \neq c_2$. Then the change*

$$\Delta H_{\text{cb}}(\xi) := H_{\text{cb}}(x \oplus \xi) - H_{\text{cb}}(x)$$

is determined entirely by the conflict-pair contributions from columns c_1 and c_2 , together with the one or two boxes containing (r, c_1) and (r, c_2) . No other term of the digit-restricted objective changes.

Proof. On the row-permutation state space, the heuristic objective is exactly H_{cb} , so only column and box contributions matter. A within-row swap preserves the row permutation, so the omitted row contribution remains zero. Every cell outside row r is unchanged, and within row r only the two swapped positions change value. Hence only the two corresponding columns can change their conflict counts. Likewise, only the boxes containing those two cells can change. All other columns and boxes see the same multiset of digits before and after the swap, so their contributions are unchanged. $\square$

Algorithm 1 Row-swap local search

Require: Puzzle with clues $\mathcal{C}$, max steps T , restarts R

```
1: for  $r = 1$  to  $R$  do
2:   Initialise each row as a permutation of  $1, \dots, 9$  consistent with its clues
3:   for  $t = 1$  to  $T$  do
4:     if no column/box conflicts then
5:       return solution
6:     end if
7:     Choose a row involved in a conflict
8:     Find the best swap of two non-clue cells in that row (largest decrease in  $H_{cb}$ )
9:     Apply the swap (or a random swap if no improving swap exists)
10:  end for
11: end for
12: return best assignment found
```

Step	Row	Swap (columns)	Swap (digits)	Type	Conflicts (before)	(after)
1	5	$7 \leftrightarrow 8$	$2 \leftrightarrow 6$	best	56	53
2	6	$3 \leftrightarrow 6$	$1 \leftrightarrow 5$	best	53	49
3	2	$2 \leftrightarrow 3$	$2 \leftrightarrow 8$	best	49	45
4	4	$7 \leftrightarrow 6$	$7 \leftrightarrow 4$	random	45	47
5	7	$1 \leftrightarrow 5$	$3 \leftrightarrow 9$	best	47	40
6	4	$6 \leftrightarrow 7$	$7 \leftrightarrow 4$	best	40	38
7	2	$2 \leftrightarrow 9$	$8 \leftrightarrow 7$	best	38	36
8	1	$3 \leftrightarrow 4$	$8 \leftrightarrow 4$	best	36	35

Table 6: First eight swap steps for the standard example puzzle (seed 0). The table records both the swapped column indices and the digits occupying those columns before each move. Rows and columns are 1-indexed.

A.2 Trace on a standard puzzle

To make the row-swap heuristic concrete, Figure 4 shows the standard example puzzle used in many Sudoku tutorials. With seed 0 (restart 0), the row-wise random initialisation used by the solver produces the filled grid in Figure 5; this state has 56 column/box conflict pairs. Table 6 shows the first few swaps from that initial state.

Step 2 illustrates the locality claim. After Step 1, row 6 contains

$$(7, 3, 1, 8, 2, 5, 4, 9, 6),$$

and the chosen move swaps the entries in columns 3 and 6, exchanging 1 and 5. By Lemma 4, only column 3, column 6, the box on rows 4–6 and columns 1–3, and the box on rows 4–6 and columns 4–6 need to be inspected. Before the swap these four units contribute

$$4 + 6 + 1 + 2 = 13$$

conflict pairs; afterwards they contribute

$$3 + 3 + 2 + 1 = 9.$$

Thus $\Delta H_{cb} = -4$, exactly as reported in Table 6. For this instance the solver reaches a full solution after 227 swaps on the same restart.

A.3 A discrete Zubarev walk

The row-swap heuristic above is tailored to the strong-snapping Sudoku setting. It is also natural to test a more generic training dynamic from the p-adic regression literature. In [13],

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 4: Standard example puzzle used for the heuristic traces.

5	3	8	4	7	2	1	9	6
6	2	8	1	9	5	3	4	7
3	9	8	1	4	7	2	6	5
8	2	9	5	6	4	7	1	3
4	5	7	8	9	3	2	6	1
7	3	1	8	2	5	4	9	6
3	6	1	7	9	5	2	8	4
2	6	3	4	1	9	7	8	5
3	2	4	6	8	5	1	7	9

Figure 5: Seed-0 row-wise random initialisation (restart 0), before any swaps.

Zubarev proposes a random-walk-based optimisation scheme for loss functions built from p -adic norms, precisely because such losses are locally constant almost everywhere and hence poorly served by gradient methods.

Concretely, the proposed training dynamics is a discrete-time random process

$$w_{t+1} = w_t + \xi_t(w_t, \beta_t), \quad (3)$$

where $\beta_t > 0$ is a “continuity” (inverse temperature) parameter and ξ_t is sampled from a distribution biased toward loss-decreasing moves. In Zubarev’s formulation the conditional law has density (with respect to the p -adic Haar measure)

$$P(\xi \mid w, \beta) \propto \exp(-\beta(L(w + \xi) - L(w))). \quad (4)$$

Under mild conditions (in particular, that the set of improving moves has nonzero measure), Zubarev shows there is a regime of β values for which the expected loss decreases, i.e. $L(w_t)$ is a strict supermartingale [13].

In the digit-restricted implementation used here, the state is already digit-valued, clue-consistent, and row-permuted. We therefore treat the Zubarev walk as a *finite-move* stochastic optimiser:

- The parameter vector w is the current Sudoku filling $x \in D^{81}$.

Algorithm 2 Zubarev walk over row swaps (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T , restarts R , schedule $\{\beta_t\}_{t=1}^T$

```
1: for  $r = 1$  to  $R$  do
2:   Initialise each row as a permutation of  $1, \dots, 9$  consistent with its clues
3:   for  $t = 1$  to  $T$  do
4:     if no column/box conflicts then
5:       return solution
6:     end if
7:     Choose a row involved in a conflict
8:     Enumerate row swaps  $\Xi_r(x)$  and compute  $\Delta H_{\text{cb}}(\xi)$  for each  $\xi \in \Xi_r(x)$ 
9:     Sample  $\xi \in \Xi_r(x)$  with probability  $\propto \exp(-\beta_t \Delta H_{\text{cb}}(\xi))$ 
10:    Apply swap  $\xi$ 
11:  end for
12: end for
13: return best assignment found
```

- The loss is the duplicated unit-scope column/box count $H_{\text{cb}}(x)$.
- A move ξ is a swap of two non-clue cells within a single row (preserving each row as a permutation of $1, \dots, 9$).

Replacing Haar measure and a continuous p -adic state space by counting measure on row swaps changes the hypotheses of Zubarev’s result. No supermartingale or convergence guarantee is claimed for the finite heuristic below; the Boltzmann weighting is used only as a motivated move-selection rule.

For a chosen row r and current state x , let $\Xi_r(x)$ be the set of all such swaps, and let $\Delta H_{\text{cb}}(\xi) := H_{\text{cb}}(x \oplus \xi) - H_{\text{cb}}(x)$ be the change in loss after applying swap ξ (where $x \oplus \xi$ denotes “apply the swap”). Replacing the Haar measure in (4) with the counting measure on the finite set $\Xi_r(x)$ gives the discrete sampling rule

$$\Pr(\xi \mid x, \beta_t) = \frac{\exp(-\beta_t \Delta H_{\text{cb}}(\xi))}{\sum_{\xi' \in \Xi_r(x)} \exp(-\beta_t \Delta H_{\text{cb}}(\xi'))}. \quad (5)$$

When $\beta_t \rightarrow 0$ this is uniform random swapping; when $\beta_t \rightarrow \infty$ it concentrates on the best (most loss-decreasing) swap, recovering a soft version of the greedy stepwise algorithm. In practice we use a simple annealing schedule: β_t increases linearly from β_0 to β_1 over the allotted steps, gradually shifting from exploration to exploitation.

A.4 Zubarev trace on the standard puzzle

Using the same standard example puzzle, the Zubarev walk (seed 0, restart 0, $\beta_0 = 0.5$, $\beta_1 = 6.0$ linear schedule) starts from the same row-consistent initialisation with 56 column/box conflict pairs. Unlike the greedy stepwise method, the walk sometimes accepts loss-increasing or loss-neutral swaps early on; Table 7 shows the first few moves. For this instance the Zubarev walk reaches a full solution after 10,029 swaps on the same restart.

B Illustrative experiments

We implemented both heuristics above in pure Python (see accompanying code). The prime was set to $p = 11$, but in the digit-restricted regime only the induced indicator behaviour matters: the search evaluates the integer conflict count H_{cb} directly and never computes a p -adic valuation numerically, so the p -adic norm routines in the code are illustrative of the construction rather than part of the solve path.

Step	Row	Swap (columns)	Conflicts (before)	(after)
1	5	2 $\leftrightarrow$ 8	56	56
2	6	7 $\leftrightarrow$ 8	56	56
3	2	3 $\leftrightarrow$ 7	56	55
4	8	2 $\leftrightarrow$ 8	55	51
5	4	2 $\leftrightarrow$ 3	51	50
6	4	6 $\leftrightarrow$ 7	50	51
7	2	7 $\leftrightarrow$ 8	51	52
8	8	3 $\leftrightarrow$ 7	52	49

Table 7: First eight Zubarev-walk swap steps for the standard example puzzle (seed 0). Rows and columns are 1-indexed.

B.1 Experimental setup

To obtain a quick, reproducible testbed, we generated random puzzles by:

1. sampling a random solved grid; then
2. *carving* it down to a specified clue count by removing entries uniformly at random.

This procedure does *not* enforce uniqueness of the resulting puzzle. Since Theorem 3 characterises the entire set of global minimisers as the set of valid completions, a puzzle with several completions has several global minimisers, and the heuristics report whichever one their trajectory reaches first. Each carved puzzle was then solved with up to $T = 60,000$ steps and $R = 15$ random restarts. All results reported below use a fixed RNG seed (123) for reproducibility. The two methods receive the same 18 puzzle seeds and the same corresponding solve seeds, so their results are paired. There is only one solve seed per method–puzzle pair; the sample is not difficulty-normalised and supports no inferential performance claim.

The table can be regenerated from the repository root with the following commands:

```
python3 code/run_experiments.py --outdir outputs/paper_stepwise \
--seed 123 --n 6 --clues 36,30,26 --max-steps 60000 \
--restarts 15 --method stepwise
python3 code/run_experiments.py --outdir outputs/paper_zubarev \
--seed 123 --n 6 --clues 36,30,26 --max-steps 60000 \
--restarts 15 --method zubarev --beta0 0.5 --beta1 6.0 \
--beta-schedule linear
```

B.2 Results

Table 8 summarises 36 method–puzzle runs over 18 paired puzzles (six puzzles at each clue count, evaluated by both heuristics). Both methods succeeded on every puzzle. The row-swap method took fewer steps on 14/18 paired runs, including 5/6, 6/6, and 3/6 puzzles at 36, 30, and 26 clues respectively. Runtimes are machine-dependent and noisy, and the sample is too small for stronger algorithmic claims.

Figure 6 shows the conflict trajectory for the stepwise row-swap heuristic on the first 30-clue table instance, with puzzle seed 115642242 (solve seed 2738956839), using the row-wise random initialiser described above, $T = 60,000$, and $R = 15$. The plotted run is restart 0, and the vertical axis is the duplicated column/box conflict count H_{cb} . The corresponding trace puzzle, solution, CSV row, and figure source are all included in `outputs/paper_stepwise/` in the submission archive.

Method	Clues	Puzzles	Solved	Median steps	Median time (s)
Stepwise	36	6	6	537.5	0.270
Stepwise	30	6	6	2784	1.315
Stepwise	26	6	6	7040	3.990
Zubarev walk	36	6	6	4533.5	2.091
Zubarev walk	30	6	6	8751.5	3.951
Zubarev walk	26	6	6	9911	4.786

Table 8: Descriptive smoke-test results on 18 paired, randomly carved puzzles (not uniqueness-checked), using $T = 60,000$ steps and $R = 15$ restarts. The Zubarev walk uses a linear schedule $\beta_0 = 0.5$ to $\beta_1 = 6.0$.

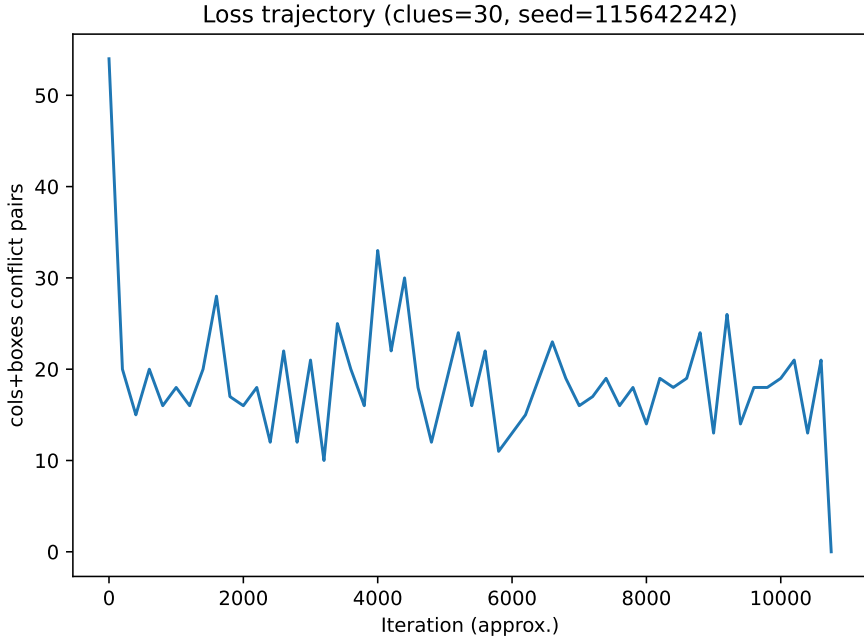


Figure 6: Conflict trajectory for the stepwise row-swap heuristic on the paired-table 30-clue instance with puzzle seed 115642242 (solve seed 2738956839), from the row-wise random initialisation on restart 0.

C Powers-of-two experiments

An alternative maps digits $d \in \{1, \dots, 9\}$ to powers of two,

$$d \mapsto 2^{d-1} \in A := \{1, 2, 4, \dots, 256\}.$$

If every cell variable is restricted to A , then the condition that a row, column, or box contains each digit exactly once is equivalent to a single sum constraint

$$\sum_{(r,c) \in g} \beta_{r,c} = 1 + 2 + \dots + 256 = 2^9 - 1 = 511,$$

for each unit g . This replaces the pairwise penalties by 27 residuals of the form

$$r_g(\beta) := 511 - \sum_{(r,c) \in g} \beta_{r,c}.$$

For a prime p , one can then study the objective

$$L_p(\beta) := \sum_g |r_g(\beta)|_p,$$

optionally together with a regulariser that encourages $\beta_{r,c} \in A$ and enforces the clues.

Archived local-search experiments with this encoding were run on 50 Euler Project Problem 96 puzzles. Table 9 summarises the outcomes. None of these runs found a valid completion.

Experiment	Setting	Solved	Summary
E1	Greedy cell-swap on 15 primes, 50 puzzles, 3 random initialisations per puzzle–prime pair (2250 runs)	0/2250	Mean lift 0.003; mean 12.5 steps
E2 greedy	Greedy cell-swap on the 5 strongest E1 primes, again with 50 puzzles and 3 initialisations (750 runs)	0/750	Mean lift 0.009; mean 19.3 steps
E2 SA	Simulated annealing on the same 750 runs	0/750	Mean lift 0.000; mean 5000 steps

Table 9: Archived local-search results for the powers-of-two encoding. The E1 prime sweep used primes 2, 3, 5, 7, 11, 13, 17, 23, 47, 73, 97, 127, 257, 521, 2311.

Funding

This research was supported by an Australian Government Research Training Program Scholarship.

Compliance with ethical standards

This article does not contain any studies with human participants or animals performed by the author.

Conflict of interest

The author declares that he has no conflict of interest.

Author contributions

This is a single-author paper. The author is responsible for the conceptual development, proofs, implementation, and writing.

Code and data availability

The Python implementation used for the heuristic experiments is publicly available at:

<https://github.com/solresol/sudoku-padic-regression>
Dataset archive: Hugging Face dataset
Experiment-code revision: `ba49a8b2e51670f66db624d814d3f31678881b93`
Main scripts: `code/padic_sudoku_regression.py`, `code/run_experiments.py`
Powers-of-two script: `archive/scripts/run_experiments.py`
Raw E1 data: `archive/results/e1_prime_sweep.csv`
Raw E2 data: `archive/results/e2_heuristic_comparison.csv`

The submitted source package includes the experiment CSV files, puzzle seeds, trace puzzle and solution files, and figure source used for Tables 8 and 9 and Figure 6. The raw-data revision above generated the archived CSV rows; the submitted driver corrects the summary formatter so that half-integer medians are not truncated. The main experiment outputs were produced with Python 3.11.6 and Matplotlib 3.9.0 on an Apple M1 system. Timing columns are retained as descriptive provenance only.

Interactive demonstration

A browser-based demonstration of the construction is shown in Figure 3 and available at

<https://padic-logic.symmachus.org>
<https://padic-logic.symmachus.org/#sudoku>

The Sudoku and Boolean-CSP compilers, residual dataframes, objective evaluations, and representative searches run client-side. Their source is in `padic-logic.symmachus.org/` in the same repository; the deployed values shown in Figure 3 were checked on 13 July 2026. The optional natural-language schema extractor requires a browser exposing the on-device Prompt API and is therefore not part of the browser-independent reproducibility claim. The complete batch results remain tied to the experiment-code revision and dataset archive above rather than to the mutable teaching site.

References

- [1] G. Baker, S. McCallum, and D. Pattinson. Linear regression in p -adic metric spaces. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 333–347 (2025). <https://doi.org/10.1134/S2070046625040016>
- [2] G. Baker. Expressivity and NP-hardness of p -adic linear regression. *p-Adic Numbers, Ultrametric Analysis and Applications* 18(2), 210–216 (2026). <https://doi.org/10.1134/S2070046626020081>
- [3] J. A. De Loera, J. Lee, S. Margulies, and S. Onn. Expressing combinatorial optimization problems by systems of polynomial equations and the Nullstellensatz. arXiv:0706.0578 (2007). <https://arxiv.org/abs/0706.0578>
- [4] T. Schiex, H. Fargier, and G. Verfaillie. Valued constraint satisfaction problems: hard and easy problems. In *Proceedings of IJCAI-95*, 631–639 (1995). <https://www.ijcai.org/Proceedings/95-1/Papers/083.pdf>
- [5] J.-C. Régim. A filtering algorithm for constraints of difference in CSPs. In *Proceedings of AAAI-94*, 362–367 (1994). <https://aaai.org/Papers/AAAI/1994/AAAI94-055.pdf>
- [6] J. A. Ellis-Monaghan and I. Moffatt. A note on recognizing an old friend in a new place: list coloring and the zero-temperature Potts model. arXiv:1406.5532 (2014). <https://arxiv.org/abs/1406.5532>
- [7] A. Lucas. Ising formulations of many NP problems. *Frontiers in Physics* 2, 5 (2014). <https://doi.org/10.3389/fphy.2014.00005>
- [8] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird. Solving large-scale constraint-satisfaction and scheduling problems using a heuristic repair method. In *Proceedings of AAAI-90*, 17–24 (1990). <https://cdn.aaai.org/AAAI/1990/AAAI90-003.pdf>

- [9] Donald E. Knuth. *Dancing Links*. *Millennial Perspectives in Computer Science*, 187–214, 2000. Also available as arXiv:cs/0011047. <https://doi.org/10.48550/arXiv.cs/0011047>
- [10] Helmut Simonis. *Sudoku as a Constraint Problem*. In *Modelling and Reformulating Constraint Satisfaction Problems*, Fourth International Workshop, Sitges (Barcelona), Spain, October 1, 2005. <https://citeseerx.ist.psu.edu/document?doi=4f069d85116ab6b4c4e6dd5f4776ad7a6170faaf>
- [11] Inês Lynce and Joël Ouaknine. *Sudoku as a SAT Problem*. In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM 2006)*, 2006. <https://people.mpi-sws.org/~joel/publications/sudoku05abs.html>
- [12] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, 86-A(5):1052–1060, 2003.
- [13] A. P. Zubarev. p -Adic polynomial regression as alternative to neural network for approximating p -adic functions of many variables. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 413–420 (2025). <https://doi.org/10.1134/S2070046625040065>
- [14] T. Mihara. p -adic linear regression for random sampling with digitwise noise. arXiv:2604.13137 (2026). <https://arxiv.org/abs/2604.13137>
- [15] T. Mihara. Generalisation of Baker’s forcing method to arbitrary prime and NP-hardness of several p -adic optimisations. arXiv:2607.06092 (2026). <https://arxiv.org/abs/2607.06092>